Модели данных

С1. Форматы данных. CSV и XML



Московский государственный технический университет
имени Н.Э. Баумана

Факультет ИБМ

дек 2024 года

Москва

Артемьев Валерий Иванович © 2024

Что такое формат данных?

Описание структуры и содержания данных в БД, файлах, сообщениях и в памяти на физическом уровне представления.

Понятие формата относится как к набору данных в целом,
так и к отдельным элементам данных.

Структура: последовательность, перечни, вложенность и связность.
Синтаксис: разделители, ограничители и разметка (tag).
Элементы данных переменной или фиксированной длины.
Формальное описание: схема данных со спецификацией типов.
Представление: расположение элементов или регулярное выражение.

Важные параметры формата:

- кодировка символов
- языковая локализация (русские метки, формат чисел и дат)
- используемые разделители и метки (тэги)
- наличие метаданных и контроля по ним

Классификация форматов данных

Основные факторы классификации форматов файлов:

- тип хранимых данных,
- степень открытости,
- внутренняя организация данных,
- назначение.

Классификация форматов по типам данных

Самая распространенная классификация по *расширениям файлов*:

Текстовые – хранят символы, бывают «*чистые*» (.txt, .md), *размеченные* (.html, .xml) или документы *со сложным форматированием* (.docx, .pdf, .odt).

Графические (изображения) – делятся на *растровые* (сетка пикселей: .jpg, .png, .bmp, .tiff) и *векторные* (математические кривые: .svg, .ai, .cdr).

Аудио – хранят *звук без сжатия* (.wav), *с сжатием с потерями* (.mp3, .aac), *сжатие без потерь* (.flac).

Видео – обычно контейнеры, внутри которых *лежат видео- и аудиопотоки, сжатые кодекам*: .mp4, .mkv, .avi, .mov.

Исполняемые и системные – содержат код для запуска программ (.exe, .dll) или служебную информацию ОС.

Классификация форматов по степени открытости

Классификация форматов файлов по степени открытости (и принадлежности):

Открытые (стандартизированные) – спецификации публичны, любой может создать программу для работы с ними: .txt, .html, .png, .svg, .odt.

Закрытые (proprietary) – привязаны к конкретному ПО и часто имеют сложную внутреннюю структуру: .psd (Photoshop), .docx (Word), .ai (Illustrator), .skp (SketchUp).

Классификация форматов по внутренней организации данных

Текстовые (plain text) – данные представлены в виде читаемых символов, их можно открыть в блокноте (.txt, .csv, .json).

Бинарные (binary) – данные закодированы в виде байтов, понятных только машине (.exe, .jpg, .mp3), для просмотра нужна специальная программа.

Контейнеры: формат описывает только «упаковку» (метаданные, структуру), а внутри может лежать что угодно, сжатое разными алгоритмами: .mkv или .mp4, куда можно «положить» видео с разными кодеками.

Классификация форматов данных по назначению

Форматы обмена / web – созданы для передачи данных между разными системами и в интернете, должны быть легкими и совместимыми:
.json, .xml, .csv, .html, .png, .jpeg.

Рабочие (проектные) – хранят «исходники» для дальнейшего редактирования со всеми слоями, эффектами и историей, они тяжелые и обычно закрытые: .psd, .aep, .prproj.

Форматы архивации / долгосрочного хранения – обычно используют сжатие без потерь или открытые стандарты, чтобы данные можно было прочитать спустя годы: .tiff, .pdf/a, .flac.

Форматы обмена данными

Форматы обмена данными – это общий язык «lingua franca» для компьютеров, с помощью *разные программы, сервисы и платформы могли общаться и понимать друг друга*.

Главные требования: *совместимость, читаемость и легкость разбора* (парсинга) форматов.

Текстовые форматы обмена (человекочитаемые) – самые универсальные, их можно прочитать в блокноте и понять структуру.

Среди них выделяются **машиночитаемые форматы обмена**.

Бинарные форматы обмена (машинно-ориентированные) – компактнее и быстрее текстовых, но нечитаемы без специальных инструментов.

Специализированные и устаревшие форматы обмена.

Текстовые форматы обмена данными

CSV (Comma-Separated Values) – простейшая таблица, где столбцы разделены запятыми (или точкой с запятой), идеален для импорта/экспорта табличных данных, но не хранит типы данных и сложную структуру.

JSON (JavaScript Object Notation) – современный стандарт для веб-API и конфигураций. Легкий, поддерживает вложенность (объекты и массивы), но до конца не стандартизирована схема (хотя есть черновики JSON Schema).

XML (eXtensible Markup Language) – более строгий и многословный «старший брат» JSON. Использует теги, поддерживает пространства имен и схемы (XSD). Популярен в enterprise-системах, SOAP-сервисах и документах (DOCX, SVG).

YAML – дружелюбный к человеку формат для конфигураций (Docker, Kubernetes, CI/CD), использует отступы вместо скобок, но чувствителен к форматированию.

Markdown – лёгкий язык разметки для документации и README, хорош для текста, но не для структурированных данных.

Машиночитаемые форматы данных

Машиночитаемые форматы данных – это форматы, в которых содержится семантика данных и данные структурированы так, что их может автоматически считывать, обрабатывать и интерпретировать компьютер без вмешательства человека.

Назначение машиночитаемых форматов данных:

Автоматизация – данные сразу попадают в базы, отчёты, аналитику.

Уменьшение ошибок – исключается человеческий фактор при вводе.

Совместимость – разные системы могут обмениваться данными без потери структуры.

Масштабируемость – удобно обрабатывать большие объёмы (тысячи строк, миллионы записей).

Распространённые машиночитаемые форматы:
JSON, XML, CSV, YAML, Parket / AVRO.

Бинарные (машинно-ориентированные) форматы обмена данными

Protocol Buffers (Protobuf) – разработан Google, требует заранее описанной схемы (.proto), компилируется в код. Очень быстрый и компактный – стандарт для gRPC и микросервисов.

Apache AVRO – используется в экосистеме Big Data (Hadoop, Kafka), хранит схему вместе с данными, что упрощает эволюцию формата.

Apache Parquet / ORC – колоночные форматы для аналитики, эффективно сжимают и позволяют читать только нужные столбцы, стандарт для Data Lake и Spark.

BSON / JSONB / MessagePack / CBOR – бинарные аналоги JSON, быстрее и компактнее, но сохраняют ту же модель данных.

Специализированные и устаревшие форматы обмена

XLS / XLSX – форматы файлов **Excel** для хранения табличных данных, используются в бизнес-аналитике и финансовой отчётности.

DBF – формат файла настольной СУБД dBase (унаследованный) для хранения и обмена данными, например, справочниками.

SQL-скрипты – текстовые файлы с командами INSERT для переноса баз данных.

EDI (Electronic Data Interchange): Строгие стандарты для бизнеса (заказы, счета), например, EDIFACT или X12. Всё ещё используются в логистике и ритейле.

INI / TOML – простые форматы конфигураций, TOML — современная альтернатива INI с явными типами.

Спецификация формата CSV



Регулярная табличная структура данных в *текстовом* файле в виде построчного представления с разделителями полей.

- Одна таблица хранится в одном файле.
- Каждая строка таблицы – это одна строка файла .
- Первая строка, как правило, содержит *имена полей (метаданные).*
- Значения полей имеют *переменную длину*.
- Разделителем значений по умолчанию служит «запятая».
- В русской локали разделитель – «*точка с запятой*» или «*табуляция*», т.к. «запятая» используется как десятичный разделитель.
- Поле, содержащее разделители, переносы строк, двойные кавычки, должно быть заключено *в двойные кавычки*.
- Символ двойной кавычки в поле должен быть удвоен.
- Значения полей могут отсутствовать, разделители – нет.
- Пустые значения в конце строки опускают вместе с разделителями.
- Существует стандарт RFC 4180.

Пример данных в формате CSV

№ п/п	Название книги	Авторы	Год выпуска	Издательство	Город	Страниц
1	XML для проектировщиков	Джеймс Бин	2004	КУДИЦ-ОБРАЗ	Москва	256
2	Регулярные выражения	Бен Форта	2005	Изд. дом «Вильямс»	Санкт-Петербург	184
3	XBRL для чайников	Чарльз Хоффман, Лив Апнесет Уотсон	2017	ООО «Альфа-книга»	Санкт-Петербург	432
4	Обработка данных на Python	Сьюзен МакГрегор	2024	БХВ-Петербург	Санкт-Петербург	432

Файл “Библиотека.CSV”

```
№ п/п;Название книги;Авторы;Год выпуска;Издательство;Город;Страниц
1;XML для проектировщиков;Джеймс Бин;2004;КУДИЦ-ОБРАЗ;Москва;256
2;Регулярные выражения;Бен Форта;2005;Изд.дом Вильямс;Санкт-Петербург;184
3;XBRL для чайников;Чарльз Хоффман,Лив Апнесет Уотсон;2017;ООО Альфа-книга;Санкт-Петербург;432
4;Обработка данных на Python;Сьюзен МакГрегор;2024;БХВ-Петербург;Санкт-Петербург;432
```

Достоинства формата CSV

- **Машиночитаемый формат:** текст, понятный компьютерам и людям, потому что содержит бизнес-метаданные.
- **Удобство** для обмена данными между приложениями, хранения данных, для представления аналитического набора данных (dataset) при машинном обучении.
- **Универсальность:** не зависит от прикладных платформ, может использоваться для различных структурированных типов данных.
- **Простота представления данных:** каждая строка – запись, значения полей – литералы, между ними разделители.
- **Совместимость:** поддержка в MS Excel, Google Sheets, Python, R, загрузчиках данных и др.
- **Компактность:** минимальная избыточность и малый размер
- **Эффективность:** быстрые чтение и запись на диск, высокая скорость обмена данными.

Недостатки формата CSV

Ограниченность структуры: одиночные таблицы с регулярным представлением ограничивают применение в сложных структурах.

Избыточность данных: справочные многократно повторяются

Скупые метаданные: в первой строке – бизнес-названия полей.

Нет явной типизации: все данные понимаются как литералы.

Ошибки разбора: неправильный разбор формата при нарушении синтаксиса, требуют дополнительной проверки и очистки данных.

Зависимость от кодировки и локали: дополнительные усилия по распознаванию кодировки и локали.

Отклонения от стандарта: много вариантов частичной поддержки стандарта или нестандартных расширений**.**

Что такое XML?



XML – спецификация, которая определяет *синтаксические правила для создаваемого языка (или формата) представления прикладных данных*.

Полученный XML–формат является *языком разметки*, т.е. использует ключевые слова (метки, тэги) для выделения элементов данных.

Поддерживает *вложенность* элементов и *контроль структуры и типов данных*.

XML обеспечивает *иерархическое и сетевое представление данных*.

Содержимое XML-документа

XML-документ включает:

1. Заголовок XML-документа
2. Пространства имён
3. Элементы и атрибуты
4. Комментарии
5. Секции CDATA.

Заголовок XML-документа

Содержит параметры обработки анализатора или специальные инструкции, размещается в начале документа, оформляется при помощи специальных тэгов:

<?xml *параметры и инструкции обработки* ?>

Используются для управления процессом разбора документа.

Пример:

version="1.0" – версия спецификации XML

encoding="windows-1251" – кодировка символов

standalone="yes" – все ссылки внутренние.

Пространства имён

Служат для обеспечения *уникальности имён*, определённых в разных приложениях, но используемых в одном приложении, например, шаблон:

```
<тэг xmlns:префикс="ссылка" …>
```

тэг – тэг корневого XML- элемента
префикс – короткое обозначение пространства имён
ссылка – uri или urn-ссылка, определяющая уникальность

Примеры именования пространств имён:

```
xmlns="urn:cbr-ru:rep101-1:v1.5.3.7"

xmlns:query="http://www.w3.org/1999/XSL/Transform"
```

Основная XML-конструкция

XML-формат состоит из *элементов*.

Основная конструкция – *контейнер*:

<Элемент>*содержимое элемента****</Элемент>***

включает:

1. *Начальный тег* ***<Элемент>***, может содержать атрибуты ***атрибут***=“*значение*”
2. *Конечный тег* ***</Элемент>***
3. *Содержимое элемента* – конкретное значение (число, дата или текст) или другие *вложенные элементы*.

Сокращенный XML-элементов

При отсутствии содержимого можно использовать *сокращенный элемент*:

<Элемент />

Сокращенный элемент может содержать атрибуты:

<Элемент атрибут="значение" />

Имена и содержимое элементов и атрибутов

Все должно быть в кодировке win-1251 или UTF-8

Имена элементов и атрибутов должны:

- начинаться *с буквы или символа подчеркивания*
- содержать *буквы и цифры, подчеркивание и точку*
- иметь длину имени *не более 32 символов.*

Элементы и значения атрибутов *не должны явно содержать символы “<” и “&”,* для них существуют специальные обозначения или специальный раздел.

Длинные строки и текст, тем более содержащий кавычки, не рекомендуется размещать в атрибутах.

Синтаксис camelCase для именования и атрибутов

Для составных имен элементов и атрибутов часто используют *способ написания camelCase* («горбатый») – слияние слов со сменой регистра букв на их стыке, например:

ВидКлиента (для элемента) или
видКлиента (для атрибута).

Альтернатива camelCase – использование точки в качестве разделителя слов:

вид.клиента

Правила оформления XML

XML-документ (сообщение) – совокупность XML-элементов, описывающих конкретные прикладные данные. При хранении в файле имеет расширение XML.

Правильно оформленный XML-документ:

- должен иметь только *один корневой элемент*
- элементы должны *вкладываться друг в друга без пересечения их контейнеров*.

Допустимый XML-документ

XML-схема – описание на языке XML, которое служит *для контроля структуры и типов данных*. Расширение файла – XSD.

Допустимый XML-документ – правильно оформленный XML-документ, который *соответсвует XML-схеме*.

Пример экземпляра XML-документа

```
<?xml version="1.0" encoding="windows-1251" ?>
<Библиотека>
  <Книга №пп="1" тема="Форматы XML" издГод="2004" страниц="256">
     <Название>XML для проектировщиков</Название
     <Издательство город="Москва"> КУДИЦ-ОБРАЗ</Издательство>
     <Автор>Джеймс Бин</Автор>
  </Книга>
  <Книга №пп="2" издГод="2005" страниц="184">
     <Название> Регулярные выражения </Название>
     <Издательство город="Москва"> Изд. дом «Вильямс» </Издательство>
     <Автор> Бен Форта </Автор>
  </Книга>
  <Книга №пп="3" издГод="2017" страниц="432">
     <Название> XBRL для чайников </Название>
     <Издательство город= "Санкт-Петербург">ООО «Альфа-книга»
     </Издательство>
     <Автор>Чарльз Хоффман</Автор>
     <Автор>Лив Апнесет Уотсон</Автор>
  </Книга>
</Библиотека>
```

XML-схема данных

Описание типа XML-документа в терминах ограничений на структуру и содержание документов, помимо синтаксических ограничений самого XML.

Эти ограничения обычно выражаются с помощью некоторой комбинации *грамматических правил и типов данных,* определяющих

- *порядок следования элементов,*
- *содержание элементов и атрибутов,*
- *уникальность и ссылочную целостность.*

Существующие языки описания XML-схем:

- Язык определения типа документа (DTD) специализированный язык описания схем, обладающий ограниченными возможностями, в частности отсутствием типов данных.
- XML-schema – язык описания структуры и содержания XML-документов (XSD) с типизацией данных.

Соглашения об именовании

При именовании элементов используются как жёсткие (конкретные), так и абстрактные имена. Имя начинается с *главной части* – существительного или словосочетания – отражает прикладную суть, далее *модификаторы* – прилагательные, существительные, глаголы – уточняют имя. В имени оставляют только одно вхождение слова, дублирующее другое слово в любой его части.

Имена должны быть *содержательными и удобочитаемыми*:

- не использовать предлоги, союзы, местоимения и артикли;
- склеивать слова с изменением регистра между ними (CamelCase–имена),
- смена регистра при наличии сокращений и цифр: **ОКАТОкод, Счёт2порядка**;
- в имени только латинские (или русские буквы), цифры и знак подчеркивания «_»;
- имена элементов с заглавной буквы**,** атрибутов – со строчной;
- имена должны быть краткими, длиной до 30 символов.
- нельзя одинаково именовать родителя и дочерний элемент, элемент и его атрибут.

Множественное число в имени употреблять в следующих случаях:

- для одиночных группировочных элементов;
- для элементов, отражающих понятия во множественном числе;
- для атрибутов со списком литералов.

Сокращения только общепринятые или принятые для этой предметной области. При сокращении словосочетания должно оставаться *хотя бы одно слово полностью*. При сокращении русских слов и словосочетаний применять ГОСТ 7.012-2011, а на английском языке – ГОСТ 7.11-2004.

Желательно вести *словарь имён*.

Рекомендуемые XML–типы данных и их ограничения

Прикладной тип данных	XML–тип	Ограничения				
		длины[1]	разрядности	диапазона	списка значений	шаблона
Текст	string	+				+
Строка/код	string/ token/ NMTOKEN	+			+	+
Идентификатор/ ссылка	ID/ IDREF	+			+	+
Номер (натуральное число)	positiveInteger		+	+	+	+
Счётчик/номер (начиная с 0)	nonNegativeInteger		+	+	+	+
Десятичное целое число	integer		+	+	+	+
Десятичное вещественное число	decimal		+	+	+	+
Логическое значение	boolean					+
Дата и время	date/ gYearMonth/ gYear/ time/ dateTime			+	+	+
Продолжительность	duration			+	+	+
Список литералов определённого типа[2]	list of types	+			+	
Список кодов	NMTOKENS	+			+	
Список ссылок	IDREFS	+			+	

[1] Или ограничения количества членов списка length, minLength, maxLength.
[2] Задаются ограничения на список, помимо ограничений на тип членов списка.

Пояснения к типам XML-данных

- Для *многострочных пояснительных текстов* используется XML–тип string, допускает символы перевода строки, табуляции, избыточные пробелы.
- Для *одиночных строк и кодов* с широким набором символов, может быть string или token – «ужатая строка» - удалены символы перевода строки, табуляции, избыточные пробелы.
- *Для справочных кодов в атрибутах* можно применять тип NMTOKEN, который допускает буквы, цифры, символы “_” (подчерк), “–” (дефис), “**.**” (точка), но не пробелы.
- *Для уникальной идентификации элементов и их связывания* – ID и IDREF, шаблон соответствует XML–имени.
- *Представление натуральных чисел* на основе типа positiveInteger.
- *Для штучного количества и номеров* (начиная с 0) – тип nonNegativeInteger.
- *Десятичное целое число (integer) и Десятичное вещественное число (decimal)* для чисел без знака «+» и лидирующих нулей.
- *Логические значения* строятся на основе XML–типа boolean, рекомендуется шаблон “[01]”, 0 – ложь, 1 – истина.
- Даты григорианского календаря и время дня по ГОСТ ИСО 8601–2001:
  – *дата* относится к типу date и представляется в формате “CCYY-MM-DD”;
  – *дата и время* типа dateTime в формате всемирного времени “CCYY-MM-DDThh:mm:ssZ”;
  – *продолжительность* имеет тип duration (“P1Y” – год, “P1M” – месяц, “P1D” – сутки).
- Списки атрибутов на основе списка XML–типов list of types, помимо ограничений на тип члена списка, существуют ограничения на количество членов списка length, minLength, maxLength.
- Список кодов и список ссылок могут в атрибутах использовать типы NMTOKENS и IDREFS. Однако, для них не доступно ограничение по шаблону.

Пример XML-схемы

```
<?xml version="1.0" encoding="Windows-1251"?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">

    <xsd:complexType name="Example_IntegerElement_Type">
        <xsd:simpleContent>
            <xsd:extension base="xsd:integer">
                <xsd:attribute name="attr1" type="xsd:integer"/>

                <xsd:attribute name="attr2" type="xsd:string"/>
            </xsd:extension>
        </xsd:simpleContent>
    </xsd:complexType>
```

Основные технологии, связанные с XML

1. SAX- и DOM-парсеры для разбора документов XML.
2. DTD-описания и схемы XML для проверки структуры документа.
3. Язык XPath и технология XSLT для преобразования документов XML в HTML, XML или текстовый формат.
4. Хранение в БД и обработка XML-документов (MS SQL, Oracle, MySQL, PostgreSQL).
5. Расширения языка SQL и Xquery для работы с XML.
6. XForms – стандарт ввода XML-данных , нестандартное решение MS InfoPath.
7. Web-сервисы и стандарты SOAP, WSDL, UDDI, язык BPEL для интеграции данных и приложений.
8. Стандарты семантического web: RDF, RDFS, OWL, SPARQL.

Средства для обработки XML

Средства подготовки XML:

- Простые редакторы текста, не MS Word, а Блокнот
- Специальные средства подготовки – XMLSpy
- Прикладные средства подготовки – АРМ ведения XML-форматов банковской отчетности

Средства разбора и контроля:

- Браузер MS Internet Explorer
- psrjdjtСредства разбора и контроля в АРМ ведения XML-форматов банковской отчетности
- Средства разбора и контроля в XMLSpy

Достоинства XML-формата

- **Машиночитаемый формат:** текст, понятный компьютерам и людям, потому что содержит бизнес-метаданные.
- **Универсальность:** не зависит от прикладных платформ, может использоваться для различных структурированных типов данных.
- **Гибкость**: позволяет создавать сложные структуры данных иерархические и ограниченно сетевые, включая вложенные элементы и атрибуты.
- **Контролируемость**: с помощью схем (XSD) можно контролировать структуру и данные, обеспечивать их целостность и соответствие определенным правилам, что важно для критических приложений.
- **Расширяемость**: легко расширяется, можно добавлять новые элементы и атрибуты без нарушения существующей структуры данных, подходит для долгосрочных проектов.
- **Стандартизация**: является международным стандартом, что обеспечивает его широкую поддержку и совместимость с различными приложениями.
- **Широкая поддержка:** наличие инструментов для редактирования, разбора, преобразования и отображения метаданных и данных, встроен во многие СУБД в качестве типа данных.

Недостатки XML-формата

- **Сложность**: имеет сложный синтаксис, что может затруднить его использование, плохая обозримость документов и метаданных.
- **Ограниченность**: контроль данных по схеме реализует лишь простые ограничения целостности данных и ссылок без проверки бизнес-правил.
- **Избыточность**: содержит много избыточной информации, такой как теги и имена атрибутов, что увеличивает объем данных и усложняет их обработку.
- **Объемность**: XML файлы могут быть большими, что увеличивает время передачи данных по сети, что критично в приложениях, где важна скорость передачи данных.
- **Производительность**: разбор XML более медленный и требует больше ресурсов по сравнению с форматами JSON или CSV, что может быть проблемой при высоких требованиях к производительности.

Моделирование XML-формата





Возможно поднять ФМД формата до ЛМД

Библиотека (Книга+)

Книга (Название,
Аннотация?,
Издательство,
Автор+)

Книга [№пп**,**
тема?,
изданиеВид?,
изданиеГод,
страниц?**]**

Издательство
[город**]**

Нотации логической модели XML-форматов

- Табличная спецификация
- Графические нотации:
  - XML-диаграммы
  - UML-диаграммы классов
  - ER-диаграммы для БД
  - Вложенность уровней элементов
  - Иерархия уровней элементов
- Языков*ы*е нотации:
  - DTD
  - XML-схема
  - Иерархическое описание
  - Описание вложенности уровней (на основе DTD)

Табличная спецификация XML-формата

Имя элемента/ атрибута	Обозначение, кратность появления	Описание элемента или атрибута	Домен / тип данных (XML-тип)

Используются следующие обозначения:

Э – элемент, А – атрибут.

Кратность появления элементов/атрибутов обозначается следующим образом:

[0..1] – элемент/атрибут является необязательным;

[1] – элемент/атрибут является обязательным;

[число] – элемент повторяется конкретное число раз;

[0..n] – элемент является необязательным, количество элементов не ограничено;

[1..n] – количество элементов не менее одного;

[мин..макс] – диапазон повторения элементов.

Табличная нотация XML-формата

Имя контейнера	Обозначение и кратность	Описание	Домен / тип данных
Данные101S ...	Э[1]	Оборотная ведомость Сбербанка по регионам	
БалансовСчета	Э[0 ..1]	Балансовые счета	
Счёт2порядка	Э[1 ..n]	Счёт второго порядка	
Актив-Пассив	А[1]	Признак актив/пассив	строка Допустимые значения: А – актив; П – пассив;
НомерСчёта	А[1]	Номер счёта 2-го порядка	счет2порТип
ВалютаОпераций	А[1]	Валюта операций	строка Допустимые значения: Рубли – в рублях; ИнВал – в инвалюте и драгметаллах; Итого – в рублях + в инвалюте и драгметаллах;
ВходОстаток	А[1]	Входящие остатки на начало отчетного периода	целое16неотриц-Лок Единица измерения: ТысРуб
ДебетОборот	А[1]	Обороты по дебету за отчетный период	целое16неотриц-Лок Единица измерения: ТысРуб
КредитОборот	А[1]	Обороты по кредиту за отчетный период	целое16неотриц-Лок Единица измерения: ТысРуб
ИсходОстаток	А[1]	Исходящие остатки на конец отчетного периода	целое16неотриц-Лок Единица измерения: ТысРуб

Диаграммы вложенности и иерархии уровней элементов XML-формата





Языковое описание XML-формата в виде иерархии уровней

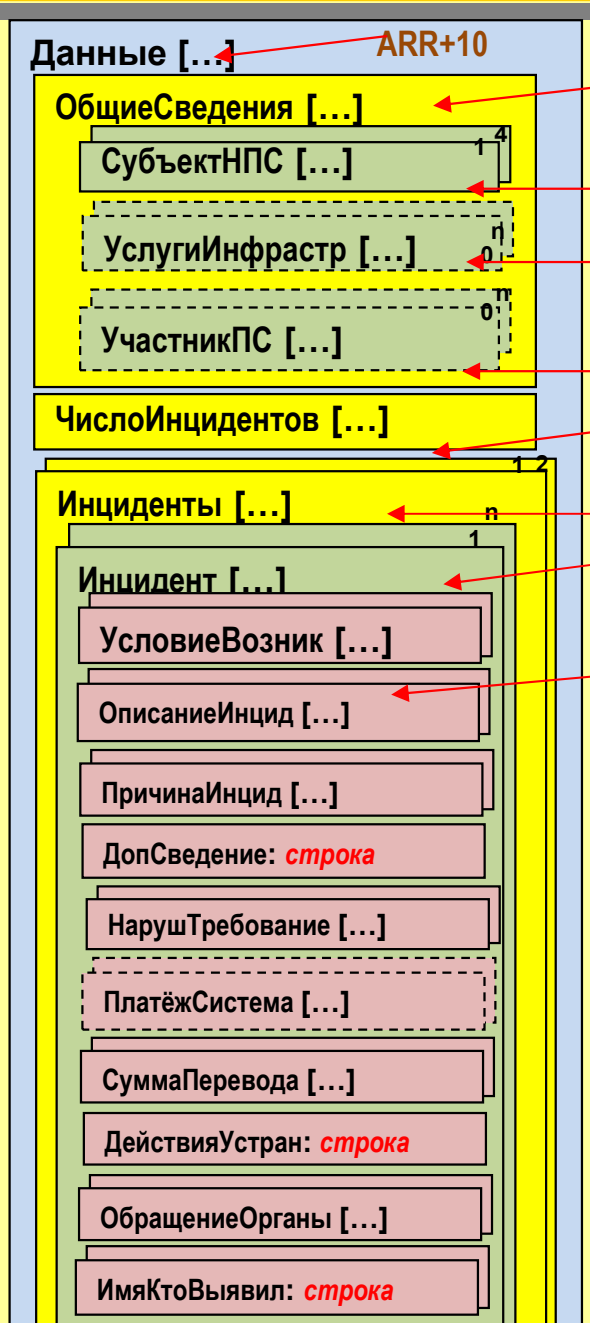
Данные101S

- **БалансовСчета?**
  - **Счёт2порядка+**
- **СчетаДоверУпр?**
  - **Счёт2порядка+**
- **ВнебалансСчета?**
  - **Счёт2порядка+,**
- **СчетаСрочСделок?**
  - **Счёт2порядка+,**
- **СчетаДепо?**
  - **Счёт2порДепо+**

Принятые обозначения

- Имена элементов выделены жирным шрифтом
- Уровень задаётся отступом
- Обозначение “?”означает повторение элемента/атрибута 0 или 1 раз
- Обозначение “+” означает повторение от 1 до n
- Обозначение “*”означает повторение от 0 до n

Пример диаграммы вложенности уровней XML-формата (форма 0403203)



ARR+10

Раздел 1

ARR+11

ARR+12

ARR+13

Раздел 2

ARR+2

Разделы 3 и 4

ARR+3 (ARR+4)

Графа 4 раздела 3 (Графа 5 раздела 4)

Графа 5 раздела 3 (Графа 6 раздела 4)

Графа 6 раздела 3 (Графа 7 раздела 4)

Графа 7 раздела 3 (Графа 8 раздела 4)

Графа 9 раздела 3 (Графа 10 раздела 4)

Графа 10 раздела 3 (Графа 11 раздела 4)

Графа 11 раздела 3 (Графа 12 раздела 4)

Графа 14 раздела 3 (Графа 15 раздела 4)

Принятые обозначения

- Элементы разных уровней показаны разными цветами
- Пунктирная рамка у необязательного элемента.
- Группа блоков обозначает их повторяемость
- Можно указать кратность появления (мин, макс)
- Для элементов со значениями определённого типа после «:» указывается тип данных
- При желании можно задавать имена атрибутов в блоках элементов

Пример языкового описания XML-формата для формы 0403203

Данные [ОКАТОкод, ОКПОкод, РНКО?, имяОператора,
почтАдрес, началоПериода, длинаПериода]

ОбщиеСведения

СубъектНПС {1,4} [роль]

УслугиИнфрастр* [регНомерОПС]

УчастникПС* [регНомерОПС]

ЧислоИнцидентов [всегоИнцид, уКлиентов, уАгентов, зарубежом]

Инциденты {1,2} [Nразд]

Инцидент+ [Nстр, кодИнцид, типИнцид, датаВыявл?, датаВозник?, датаЗаверш?, регионИнцид, нарушСроков, оценкаУбытков, БПагентКод?]

УсловиеВозник {1,12} [кодУсловия]

ОписаниеИнцид {1,9} [кодОписания]

ПричинаИнцид {1,14} [кодПричины]

ДопСведение: *строкаНепустая*

НарушТребование+ [кодТребования]

ПлатёжСистема* [операторПС]

СуммаПеревода+ [суммаРуб]

ДействияУстран: *строкаНепустая*

ОбращениеОрганы {1,7} [кодОбращения]

ИмяКтоВыявил: *строкаНепустая*

Принятые обозначения

Имена элементов выделены жирным шрифтом
Обозначение {1,4} задает повторение от 1 до 4 раз, {2} – повтор 2 раза
Обозначение “+” означает повторение от 1 до n
Обозначение “*”означает повторение от 0 до n
Обозначение “?”означает повторение элемента/атрибута 0 или 1 раз
Наличие “:” после имени элемента указывает на тип вложенного содержания, например, *строка*
В скобках указаны списки имен атрибутов

Средства структурирования в XML

- Жёсткие (конкретные) ИЛИ абстрактные имена контейнеров

  Пример для ф.101:
  2 тысячи жёстких имен элементов для счетов 2-го порядка Плана счетов бухгалтерского учёта ИЛИ
  2 абстрактных элемента (**Счёт2порядка** и **Счёт2порДепо**)

- Вертикальная структура (преимущественная ориентация на элементы) ИЛИ горизонтальная структура (ориентация на атрибуты)

- Иерархические модели ИЛИ плоские модели данных

Пример разработки XML-формата отчётности

Форма 116. Сведения о ценных бумагах, приобретенных КО

Наименование эмитента	Номер (код) эмитента	Вид деятельности эмитента	Тип ценных бумаг	Номер выпуска ценных бумаг	Количество ценных бумаг, штук	Стоимость ценных бумаг, тыс. руб.
Банк “Зенит”	3255	65	BON3	40503255B	5000.0000	52159.000
ОАО "РЖД"	1037739877295	35 60 74	BON4	4-12-65045-D	12501.0000	13976.118
ОАО "РЖД"	1037739877295	35 60 74	SHS3	1-01-55038-E	35390.0000	58.889
VTB Capital S.A.	HP	67	BON7	XS0372322460	100.0000	3929.379
VTB Capital S.A.	HP	67	BON7	XS0372322461	320.0000	12345.678
ЗПИФ "Афина Реалти" УК"ПФТ"		65	BON8	RU000A0JQC49	595.0000	29994.000

Типы эмитентов:

КО–резидент, регистрационный номер КО
Организация–резидент (не КО), ОГРН
Банк–нерезидент, SWIFT–код
Организация–нерезидент
Инвестиционный фонд

Вариант 1. Данные по строкам с жёсткими именами в вертикальной структуре

Данные116
- **ЦеннаяБумага+**
  - **ЭмитентКОрезидент |**
    - **РегНомерКО**
    - **Имя**
    - **ВидДеят+**
  - **ЭмитентОргРезидент |**
    - **ОГРН**
    - **Имя**
    - **ВидДеят+**
  - **ЭмитентБанкНерезид |**
    - **SWIFTкод,**
    - **Имя**
    - **ВидДеят***
  - **ЭмитентОргНерезид |**
    - **Имя**
    - **ВидДеят***
  - **ЭмитентИнвестФонд**
    - **Имя**
    - **ВидДеят**
  - **ТипЦенБумаги**
  - **НомерВыпуска**
  - **ЧислоБумаг**
  - **РубСтоимость**

Характеристики формата варианта 1

- Иерархическая модель данных
- Вертикальная структура (данные преимущественно в элементах)
- Альтернативы для эмитентов
- Жесткие (конкретные) имена элементов для эмитентов
- Уровней вложенности 3
- Различных элементов 16
- Различных атрибутов 0
- Тэгов в примере 112 (2,6)
- Значений в примере 43

Жёсткие имена лучше передают семантику предметной области, «усиливают» контроль. Вертикальная структура наиболее гибкая, легко расширяемая, допускает альтернативы.

Жёсткие имена существенно увеличивают трудоёмкость описания и объем XML-данных, всё равно приходится контролировать по справочникам. Вертикальная структура создаёт много тэгов.

Самое громоздкое представление данных.

Вариант 2. От жестких имён элементов к абстрактным именам

Данные
- **ЦеннаяБумага+**
  - **ТипЭмитента**
  - **КодЭмитента?**
  - **ИмяЭмитента**
  - **ВидДеятЭмитента+**
  - **ТипЦенБумаги**
  - **НомерВыпуска**
  - **ЧислоБумаг**
  - **РубСтоимость**

Характеристики формата варианта 2
- Плоская модель данных
- Вертикальная структура (элементы без атрибутов)
- Абстрактные имена элементов для эмитентов
- Уровней вложенности 2
- Различных элементов 10
- Различных атрибутов 0
- Тэгов в примере 108 (2,3)
- Значений в примере 47

Абстрактные имена могут существенно сократить трудоёмкость описания и объём XML-данных. Плоская модель уже лучше проглядывается.

Абстрактные имена требуют введения дополнительных признаков, затрудняют контроль.
Всё еще большой объём данных.

Вариант 3. От вертикальной структуры к горизонтальной

Данные

ЦеннаяБумага+ [ТипЭмитента, КодЭмитента, ИмяЭмитента, ВидыДеятЭмитента, ТипЦенБумаги, НомерВыпуска, ЧислоБумаг, РубСтоимость]

Характеристики формата варианта 3

- Плоская модель данных
- Горизонтальная структура (нижние уровни элементов переводятся в атрибуты)
- Списковый атрибут
- Абстрактные имена контейнеров
- Уровней вложенности 1
- Элементов 2
- Атрибутов 8
- Тэгов в примере 53 (1,13)
- Значений в примере 47

Атрибут – список
ВидыДеятЭмитента: NMTOKENS
(для кодов)

Горизонтальная структура существенно сокращает объём XML-данных.
Плоская модель данных представлена явно.
Эффективный формат при небольшой избыточности данных.

Горизонтальная структура хуже передаёт связь полей и менее гибкая.
Сохраняется избыточность данных.

Вариант 4. Выделение справочника и связывание с ним

Данные

ЦеннаяБумага+ [СсылкаЭмит, ТипЦенБумаги, НомерВыпуска, ЧислоБумаг, РубСтоимость]

Эмитент+ [ИдЭмитента, ТипЭмитента, КодЭмитента, Имя, ВидыДеят]

Характеристики формата варианта 4

- Плоская модель данных с выделенным справочником эмитентов
- Проверка уникальности значений идентификатора типа ID
- Целостность ссылок типа IDREF на идентификаторы
- Горизонтальная структура (данные в атрибутах)
- Абстрактные имена контейнеров
- Уровней вложенности 1
- Элементов 3
- Атрибутов 10
- Тэгов в примере 60 (1,25)
- Значений в примере 48

Атрибуты-идентификаторы

ИдЭмитента: ID (например: Э2)

Атрибут–ссылка на идентификатор

СсылкаЭмит: IDREF

Выделение справочника и организация связей приближает формат к базе данных. Проверка уникальности идентификаторов. **Эффективный формат при большой избыточности данных**.

Для связи данных нужны дополнительные затраты на описание и представление идентификаторов и ссылок.

Вариант 5. Иерархия с выделением контекста

Данные

Эмитент+ [ТипЭмитента, КодЭмитента, Имя, ВидыДеят]

ЦеннаяБумага+ [ТипЦенБумаги, НомерВыпуска, ЧислоБумаг, РубСтоимость]

Характеристики формата варианта 5

- Иерархическая модель данных с выделенным контекстом эмитента
- Горизонтальная структура (данные в атрибутах)
- Абстрактные имена контейнеров
- Уровней вложенности 2
- Элементов 3
- Атрибутов 8
- Тэгов в примере 54 (1,35)
- Значений в примере 40

Иерархическая модель с выделением контекста позволяет снизить избыточность без создания связей за счёт вложенности. **Минимальная избыточность данных. Представление данных наиболее соответствующее концепции XML**.

Вложенность несколько увеличивает размер XML-данных.
Генерация и загрузка XML-данных с иерархической моделью более трудоёмка, чем данных с плоской моделью.

Сравнение вариантов XML-форматов

Вариант XML-формата	Уровней вложенности	Элементов / атрибутов	Элементов и атрибутов	Тэгов в примере	Значений в примере	Всего тэгов и значений	Тегов на значение
1. Иерархическая модель в вертикальной структуре с жёсткими именами	3	16 / 0	16	112	43	155	2,60
2. Плоская модель данных в вертикальной структуре с абстрактными именами	2	10 / 0	10	108	47	155	2,30
3. Плоская модель данных в горизонтальной структуре с абстрактными именами	1	2 / 8	11	53	47	100	1,13
4. Плоская модель данных в горизонтальной структуре с абстрактными именами и выделенным справочником	1	3 / 10	13	60	54	120	1,22
5. Иерархическая модель с выделением контекста, горизонтальная структура с абстрактными именами	2	3 / 8	8	54	40	94	1,35

Рекомендации по выбору дизайна XML-формата

- Предпочтительный дизайн формата для отчётности:
  - **Плоская модель с горизонтальной структурой и абстрактными именами** контейнеров ИЛИ
  - **Иерархическая модель с горизонтальной структурой и абстрактными именами**
- Для снижения избыточности:
  - **Выделение справочников и организация связей** в плоской модели ИЛИ
  - **Выделения контекста** в иерархической модели
- Для повторяющихся значений кодов в атрибутах:
  - Применять **атрибуты-списки типа NMTOKENS**
- Применение вертикальной структуры:
  - **Элементы только для повторяющихся значений**, когда нельзя (трудно) применить атрибут-список, а также для многострочного текста.

Рекомендации по именованию контейнеров и типов

- Удобочитаемые имена из нескольких слов
- Имена CamelCase (верблюдоподобные) – склеивание слов с изменением регистра между словами:
  - с заглавной буквы имена контейнеров, например: **БалансовыйСчет**
  - с прописной буквы прикладные типы, например: **строкаНепустая**
  - смена регистра при наличии сокращений и цифр в именах, например: **ОКАТОкод**, **Счёт2порядка**
- В имени хотя бы одно слово полностью
- Не использовать аббревиатуры, кроме общепринятых
- Опускать предлоги и союзы
- Длина имени не более 32 символов
- Не применять одинаковые имена для элементов разных уровней, для элементов и атрибутов

Рекомендации по контролю структуры и содержания XML-данных

- При наличии справочников не применять жёсткие имена элементов, перечисления допустимых значений и шаблоны кодов.
- Для контроля уникальности элементов использовать идентификаторы типа ID.
- Для контроля связи элементов использовать идентификаторы типа ID и ссылки IDREF.
- Шаблоны использовать для контроля длины и цифрового содержания абстрактных атрибутов с разными форматами кодов.

Терпения и удачи всем, кто связан с моделированием данных

Спасибо за внимание!

Валерий Иванович Артемьев

МГТУ имени Н.Э. Баумана, кафедра ИУ-5

Тел.: +7(985) 365-79-86
e-mail: viart@bmstu.ru